

Poker Hand Classification Using Machine Learning
A Comparative Study of Feature Methods and Models

Camerin Casesa (A25323900)

Lily Lockwood (A25320973)

Azaria Reed (A25346360)

Diana De Santiago (A25333016)

Department of Computer Science, University of Alabama in Huntsville

CS 488-01: Intro to Big Data Computing

Dr. Vineetha Menon

April 14, 2026

Table of Contents

Introduction.....3

Big Data Context Using the 5V Model..... 3

Dataset Description and Problem Overview..... 3

Hypothesis Formulation..... 3

Discussion of Methods..... 3

Data Preprocessing..... 3

Feature Methods..... 4

Feature Engineering (Domain-Based Features).....4

Feature Extraction Using One-Hot Representation..... 4

Dimensionality Reduction Using LDA.....4

Classification Models (Three Experts).....5

Linear Support Vector Machine (Linear SVM).....5

Radial Basis Function Support Vector Machine (RBF SVM)..... 5

Tree-Based Models (XGBoost and Random Forest).....5

Results, Data Visualization, and Analysis.....6

Class Distribution and Feature Correlation Analysis..... 6

LDA Projection and t-SNE Visualization of the One-Hot Feature Space..... 6

Model Performance Comparison..... 7

Baseline Feature Performance.....7

LDA Feature Performance.....8

One-Hot Feature Performance..... 8

Confusion Matrix Analysis..... 9

Performance Metrics Analysis (Accuracy and Error Analysis)..... 10

Conclusion..... 10

Evaluation of Hypothesis.....10

Key Findings and Insights.....10

Human Contribution vs. AI Contribution..... 11

Limitations and Future Improvements..... 11

References..... 12

Appendix..... 13

Introduction

Big Data Context Using the 5V Model

Examination of the Poker Hand dataset with the 5V Model—volume, velocity, variety, veracity, and value—confirms it is indeed Big Data. Incipently, its volume is evident; the dataset contains multiple records that notably cover sundry combinations of poker hands. Although the dataset does not expand in real time, it intentionally includes samples for which rapid hand analysis would be essential, underscoring its potential for high velocity in real-world use. Also, the dataset is multidimensional, with different feature types, such as categorical suit values and ordinal rank values, demonstrating its variety. The dataset is clean and well-structured, and its source, the UCI Machine Learning Repository, is reliable because a reputable academic institution maintains it and researchers frequently use its datasets, confirming its veracity. And most relevantly, the dataset supports pattern discovery and the classification of poker hands, offering real value and making it useful for both testing and benchmarking machine learning techniques.

Dataset Description and Problem Overview

The UCI Machine Learning Repository's Poker Hand dataset by R. Catral and F. Oppacher provides structured data on five-card poker hands. Each sample has 10 features: five suit values and five rank values for each card in a hand. Additionally, the target variable indicates the type of poker hand, such as a high card, pair, or flush.

The main challenge, however, is actually translating such categorical and ordinal features into the correct poker hand category. For classification to occur, a model must learn patterns in both ranks and suits. But since poker hands are combinatorial, individual cards are not the sole concern, meaning relationships among features are nonlinear, with more complex interactions.

Hypothesis Formulation

If statistical and machine learning techniques analyze card combinations, then they can accurately classify poker hands, because the frequency and structure of card ranks and suits follow consistent probabilistic patterns.

This hypothesis assumes that patterns in card combinations are not random but instead reveal predictable patterns that machine learning models can recognize. By exploring different feature representations and classification methods, it is possible to evaluate whether these patterns are recognizable to achieve high classification accuracy.

Discussion of Methods

Data Preprocessing

The Poker Hand dataset required minimal cleaning due to its structured and complete format. There were no missing values, errors, or duplicate records, so every record remained valid for analysis.

Initially, feature measurements and class labels split the dataset. Each sample contained 10 features, corresponding to five cards in a hand, with each card having two components: suit and rank. The target variable revealed the poker hand category.

To ensure no data leakage for a fair evaluation, an 80/20 stratified split divided the dataset into training and test sets before any resampling or transformations. Stratification also preserved the original class distribution in both sets.

A key preprocessing challenge involved a significant imbalance in the dataset. The majority of samples belonged to common poker hands, such as “nothing” or “one pair,” while rarer hands were much less frequent. To address this issue, the Synthetic Minority Oversampling Technique (SMOTE) balanced the class distribution only on the training set, ensuring that models could learn patterns from both common and rare classes and improving overall classification performance.

Standardization handled feature scaling. The training data fit the scaler, which later applied to the testing data, preventing the use of information from the testing set during training.

Feature Methods

Multiple feature representations participated in evaluating how different transformations affect model performance. These methods included domain-based feature engineering, one-hot encoding for categorical features, and dimensionality reduction using Linear Discriminant Analysis (LDA). Comparing model performance across these methods revealed clear differences in both the representations and models.

Feature Engineering (Domain-Based Features)

Domain knowledge purposefully created additional features that summarize important structural properties of poker hands. Specifically, the model used two derived features that capture meaningful patterns in rank distributions.

- **Unique Rank Count:** the number of distinct card ranks in a hand
- **Rank Range:** the difference between the highest and lowest card ranks

These features help represent patterns such as repetition and spread. Notably, however, because these features were actually computations from the input cards and did not reference card labels, they did not introduce data leakage.

Feature Extraction Using One-Hot Representation

The original dataset uses numbers to represent suits and ranks, but these numbers are merely indicators of categories, not sizes or orders; one-hot encoding addressed this.

Each categorical value became a binary vector, expanding the feature space to a higher dimension. This higher-dimensional representation was particularly beneficial for models like Support Vector Machines, enabling them to actually grasp the differences between categories without assuming false numerical relationships.

The training data fit the encoder, which later applied to the testing data to maintain proper separation between training and evaluation.

Dimensionality Reduction Using LDA

Linear Discriminant Analysis (LDA) reduced the dimensionality of the dataset while preserving information that separated classes. As reviewed, LDA projects data into a lower-dimensional space to maximize separation between classes.

The number of classes determined the number of LDA components, ensuring that the transformation remained valid. Only the scaled training data and corresponding labels participated in training the LDA model, which then applied to the test data.

LDA's inclusion enabled evaluating whether a compressed representation of the data could retain sufficient information for accurate classification.

Classification Models (Three Experts)

Three classification model categories underwent evaluation, each representing a different learning approach: linear methods, nonlinear kernel methods, and ensemble tree-based methods.

Linear Support Vector Machine (Linear SVM)

The Linear SVM models data using a linear decision boundary. It attempts to separate classes by finding the optimal boundary, or hyperplane, that maximizes separation, or the margin, between them.

Relevantly, this model performs well when the data is approximately linearly separable but struggles when class boundaries are complex or nonlinear.

Radial Basis Function Support Vector Machine (RBF SVM)

The RBF SVM extends the linear model by using a nonlinear kernel, enabling it to capture complex relationships by mapping the data to a higher-dimensional space.

Because poker hands involve combinations and interactions among features, the RBF kernel provides the flexibility necessary to model these nonlinear patterns effectively.

Tree-Based Models (XGBoost and Random Forest)

Tree-based models learn decision rules by repeatedly splitting the data based on features that best separate classes, forming a tree-like structure of decisions. Tree-based models naturally capture nonlinear relationships and feature interactions.

- **Random Forest:** builds multiple division trees and averages their predictions to improve stability and reduce overfitting
- **XGBoost:** builds trees sequentially, where each new tree corrects the errors of previous ones, leading to high performance optimization

Applicably, both models handle categorical-like features well and do not require strict feature scaling, qualifying them as strong models for comparison.

Results, Data Visualization, and Analysis

Class Distribution and Feature Correlation Analysis

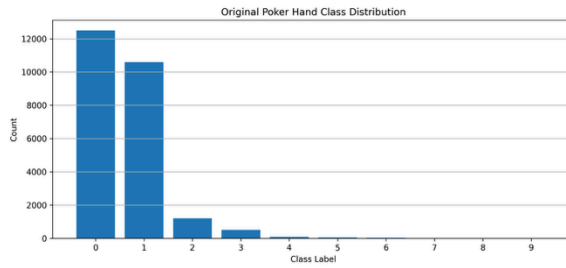


Figure 1. Original Poker Hand class distribution.

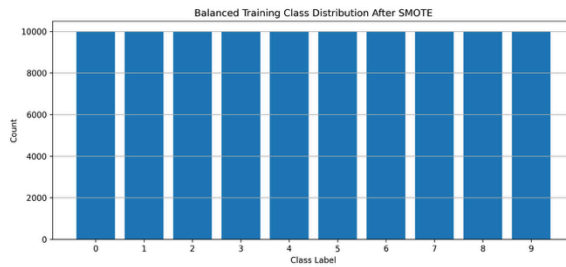


Figure 3. Balanced Poker Hand training class distribution after SMOTE.

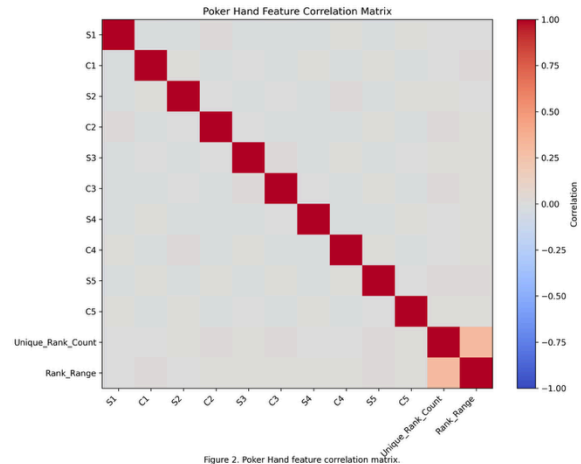


Figure 2. Poker Hand feature correlation matrix.

Figure 1 shows the original class distribution of poker hands, which remained heavily imbalanced. Most samples concentrated in the lower-ranked classes, while higher-ranked hands were very rare. This imbalance indicated that a model trained on the original data would likely favor the majority classes. Figure 3 shows the class distribution after applying SMOTE, where all classes became evenly represented. This transformation created a balanced training set, improved the model's ability to learn patterns across all classes, and reduced bias toward dominant categories.

Figure 2 presents the feature correlation matrix. As seen in Figure 2, most features exhibited very low correlation with one another, indicating minimal redundancy among the suit and rank attributes. This pattern suggested that each feature contributed distinct information to the dataset. However, the absence of strong linear relationships also suggested that simple linear models struggled to fully capture the data's underlying structure, supporting the use of more advanced techniques in later analysis.

LDA Projection and t-SNE Visualization of the One-Hot Feature Space

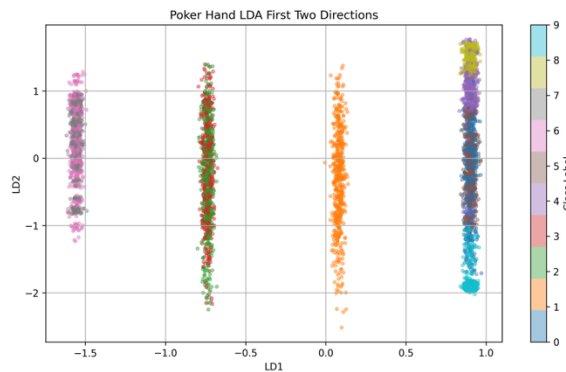


Figure 4. First two LDA directions for Poker Hand training samples.

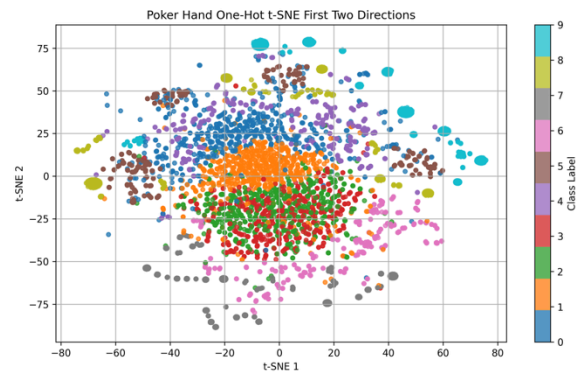


Figure 5. First two t-SNE directions from one-hot Poker Hand training samples.

Figure 4 shows how implementing LDA transformed the original feature space into a lower-dimensional representation that emphasized class separability. The dataset produced two

linear discriminant components: LD1 and LD2. As seen in Figure 4, the data formed distinct vertical clusters along LD1, with minimal variation in LD2, indicating that LD1 contained most of the discriminative information and separated broad groups of poker hand classes. However, the overlapping points within each vertical band indicate that LDA did not fully separate all classes, aligning with the lower classification accuracy observed for LDA-based models.

Unlike LDA, t-SNE provided a non-linear dimensionality reduction technique that preserved local structure rather than maximizing global class separation. Figure 5 shows that the data points formed loosely defined clusters with noticeable overlap between classes. While clear global boundaries did not emerge, as in Figure 4, local groupings of similar samples did, reflecting the complex, non-linear relationships in the original feature space. Models such as Random Forest and RBF SVM captured these relationships more effectively, resulting in higher classification accuracy.

Model Performance Comparison

Figures 6-8 depict clear differences in how the models responded to feature representations and data structure. Tree-based models, particularly Random Forest and XGBoost, consistently achieved the highest accuracies across all settings. Additionally, the RBF SVM performed strongly, especially with one-hot encoding, indicating that nonlinear methods handle the dataset well. In contrast, the Linear SVM performed significantly worse, suggesting that the data was not linearly separable. Overall, feature representation was critical, as performance varied widely across raw, reduced, and encoded features. Nevertheless, models that capture nonlinear relationships and preserve full information performed best.

Baseline Feature Performance

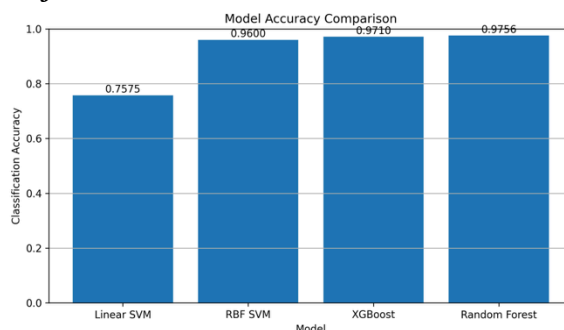


Figure 6. Model accuracy comparison on baseline scaled features.

Figure 6 depicts that, with baseline features, Random Forest and XGBoost achieved the highest accuracies, both exceeding 97%. The RBF SVM also performed well, achieving approximately 96%, confirming the presence of nonlinear relationships in the data. Linear SVM performed much worse, at around 76%, which showed that a simple linear boundary could not effectively separate the classes. These results indicate that, even without complex transformations, the dataset contains strong predictive patterns, but the models used must be capable of capturing feature interactions, such as combinations of cards and suits. Notably, this baseline served as a strong reference point for evaluating the impacts of the other feature methods.

LDA Feature Performance

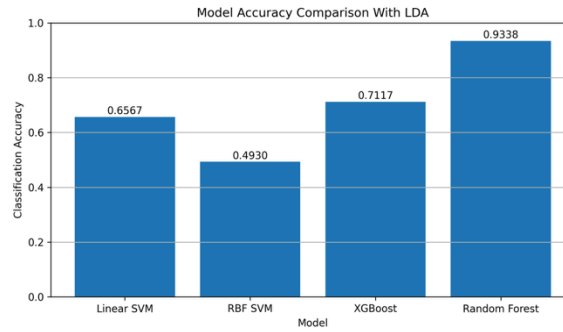


Figure 7. Model accuracy comparison with LDA features.

LDA-based features led to a noticeable drop in performance across models. Both linear and RBF SVMs declined significantly, with RBF SVM falling to around 49%, indicating severe information loss. XGBoost also dropped substantially, showing that even powerful models struggled with the reduced feature space. Random Forest maintained a relatively strong performance at approximately 93%, but it still underperformed its baseline. This decline occurred because LDA compressed the data into a small number of dimensions, thereby removing important structural information. As a result, LDA did not preserve the complex relationships required for accurate poker hand classification.

One-Hot Feature Performance

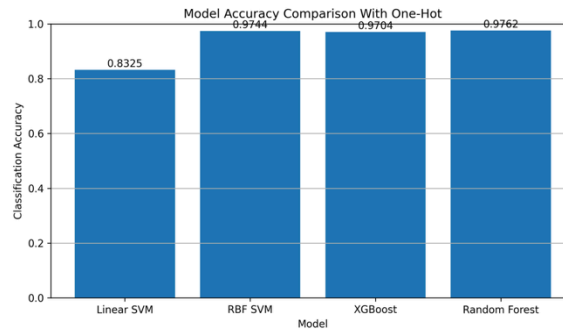


Figure 8. Model accuracy comparison with one-hot encoded features.

One-hot encoding produced the most consistent and strongest results across models. Random Forest achieved the highest accuracy, slightly improving upon its baseline performance, while RBF SVM also reached peak performance near 97%. XGBoost remained highly accurate and stable, demonstrating robustness to changes in feature representations. Linear SVM improved significantly over the baseline, rising from around 76% to over 83%, indicating that one-hot encoding helped the linear model better interpret categorical data. This improvement occurred because one-hot encoding removed misleading numerical relationships between categories. Overall, one-hot encoding preserved full feature information and enabled models to learn more accurate decision boundaries.

Confusion Matrix Analysis

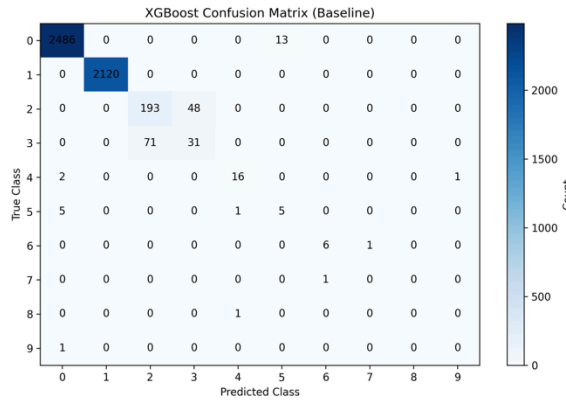


Figure 9. XGBoost confusion matrix using the best version (Baseline).

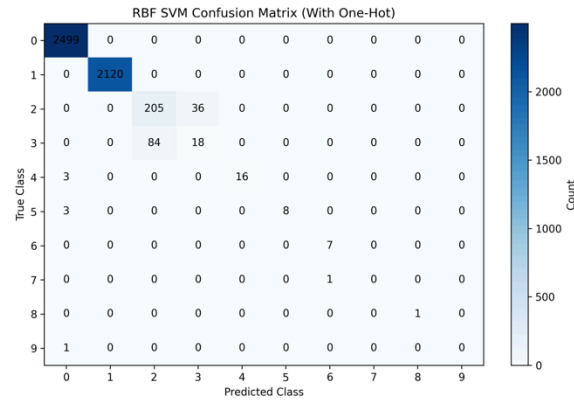


Figure 10. RBF SVM confusion matrix using the best version (With One-Hot).

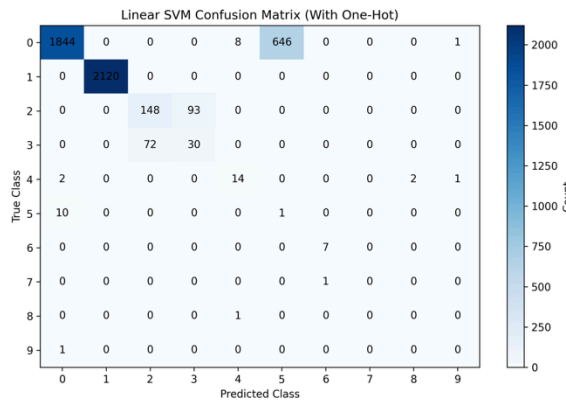


Figure 11. Linear SVM confusion matrix using the best version (With One-Hot).

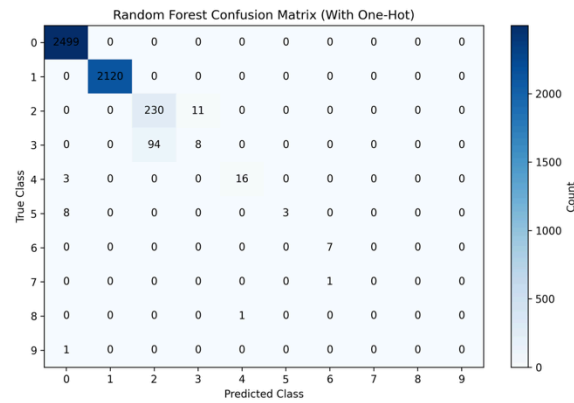


Figure 12. Random Forest confusion matrix using the best version (With One-Hot).

Figures 9-12 show the confusion matrices for each tested model using the feature representation that gave the best performance for that model—baseline, LDA, or one-hot—based on classification accuracy. Each test set contained 5002 samples. Random Forest achieved the best performance with one-hot encoding, with 119 misclassifications. RBF SVM with one-hot encoding came in second, with 128 misclassifications. XGBoost came in third, with 145 misclassifications. Linear SVM with one-hot encoding achieved the worst performance, with 838 misclassifications. The confusion matrices clearly highlight Linear SVM as an outlier, as its inability to capture complex feature interactions led to lower classification accuracy than the other models. Additionally, XGBoost stood out as another outlier, though in another category, performing best with the baseline feature set rather than one-hot encoding, indicating that it already captured useful structure from the original features. The improvement from baseline to one-hot encoding for both Random Forest and RBF SVM was small but consistent, indicating that these models benefited from richer feature representations. Overall, the results show that model performance depended more on the ability to capture non-linear relationships and feature interactions than on dimensionality reduction, as LDA consistently reduced accuracy across models.

Performance Metrics Analysis (Accuracy and Error Analysis)

Model	Classification Accuracy
Linear SVM	0.7574970011995200
RBF SVM	0.960015993602559
XGBoost	0.9710115953618550
Random Forest	0.975609756097561
Linear SVM With LDA	0.6567373050779690
RBF SVM With LDA	0.49300279888044800
XGBoost With LDA	0.7117153138744500
Random Forest With LDA	0.9338264694122350
Linear SVM With One-Hot	0.8324670131947220
RBF SVM With One-Hot	0.9744102359056380
XGBoost With One-Hot	0.9704118352658940
Random Forest With One-Hot	0.9762095161935230

Table 1. Model accuracy summary across baseline, LDA, and one-hot feature settings.

Table 1 summarizes the classification accuracy across different models and feature representations. In the baseline setting, Random Forest achieved the highest accuracy (97.56%), followed closely by XGBoost (97.10%) and RBF SVM (96.00%), while Linear SVM performed significantly lower (75.75%). These results indicate that non-linear models captured the underlying patterns in the data more effectively than linear approaches.

After applying LDA, classification performance decreased across most models. Linear SVM dropped to 65.67%, and RBF SVM declined substantially to 49.30%. XGBoost also decreased to 71.17%, while Random Forest retained relatively strong performance at 93.38%, though still lower than its baseline accuracy. These results suggest that the dimensionality reduction introduced by LDA removed important discriminative information and limited the models' ability to fully separate the classes.

In contrast, the one-hot-encoded feature representation produced the best overall performance. Random Forest achieved the highest accuracy (97.62%), with RBF SVM (97.44%) and XGBoost (97.04%) also performing well. Linear SVM improved over its baseline and reached 83.25%. These results indicate that preserving the full feature space via one-hot encoding enabled models to better capture complex relationships in the data, thereby improving classification accuracy.

Conclusion

Evaluation of Hypothesis

The hypothesis stated that if statistical and machine learning techniques analyze card combinations, then they can accurately classify poker hands, because the frequency and structure of card ranks and suits follow consistent probabilistic patterns. The results support this hypothesis. The models analyzed card combinations effectively and achieved high classification accuracy, reaching up to 97.6% during testing.

Key Findings and Insights

Key findings reveal that feature representation drove model performance more than model selection, with nonlinear models leveraging richer representations to achieve near-optimal performance. The combination of Random Forest and One-Hot encoding emerged as the most

effective configuration, achieving the highest recorded accuracy and demonstrating that preserving full categorical structure enables models to exploit complex interactions among card attributes. In contrast, Linear SVM consistently underperformed, confirming that linear decision boundaries cannot capture the inherently nonlinear relationships embedded in card combinations.

The results further show that dimensionality reduction through LDA systematically degraded performance across all models, indicating that even well-structured projections can discard critical discriminative information in combinatorial domains. Conversely, One-Hot encoding consistently improved performance across models, except for XGBoost, which had already extracted sufficient structure from the baseline features.

Overall, these findings demonstrate that accurate poker hand classification depends on preserving the full combinatorial feature space and enabling nonlinear learning, rather than compressing or simplifying the data. This study highlights a broader insight: in structured, rule-based domains, maintaining representational fidelity often outweighs algorithmic complexity, challenging the common assumption that more advanced transformations always lead to better performance.

Human Contribution vs. AI Contribution

The team contributed through designing the pipeline, selecting features and models, and interpreting the results. To elaborate, the team defined the objective of the poker hand classification task, chose meaningful feature representations, and selected appropriate machine learning models to evaluate. Further, the team analyzed outcomes and determined which approaches produced the strongest performance.

In contrast, the models contributed by learning patterns and scaling predictions. More precisely, they identified complex relationships within the data and generated accurate predictions across large datasets. Also, the models enabled consistent evaluation across multiple configurations, facilitating performance comparisons and final model selection.

Limitations and Future Improvements

This project demonstrated that machine learning models can classify poker hands based on structured patterns in card data, but it remained limited by the use of only limited domain-specific features. The feature set did not capture more advanced poker-specific structures, such as flushes and straights, because the study prioritized simpler, generalizable feature representations to enable fair comparisons across methods. The approach also used a single train-test split, which limited robustness and reproducibility.

Future improvements could include adding richer domain-specific features such as flush indicators, rank frequency counts, and straight detection to better capture the underlying structure of poker hands. The project could also implement cross-validation methods, such as k-fold, to improve reliability and ensure more consistent performance evaluation. Expanding the evaluation metrics beyond classification accuracy would further strengthen the analysis and provide deeper insight into model performance.

References

Catral, R., & Oppacher, F. (2002). *Poker hand* [Data set]. UCI Machine Learning Repository.
<https://doi.org/10.24432/C5KW38>

Appendix

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

from imblearn.over_sampling import SMOTE
from sklearn.manifold import TSNE
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# Load the Poker Hand dataset from a CSV file and rename every column.
def LoadPokerHandDatasetFromCsvFile(csvFileName):
    pokerHandDataFrame = pd.read_csv(csvFileName, header=None)
    pokerHandDataFrame.columns = [
        "S1", "C1", "S2", "C2", "S3", "C3", "S4", "C4", "S5", "C5", "CLASS"
    ]
    return pokerHandDataFrame

# Split the full table into feature columns and class labels.
def SplitFeatureColumnsAndClassLabels(pokerHandDataFrame):
    featureMeasurementGrid = pokerHandDataFrame.drop("CLASS", axis=1)
    classLabelSeries = pokerHandDataFrame["CLASS"]
    return featureMeasurementGrid, classLabelSeries

# Build domain knowledge features.
def BuildDomainKnowledgeFeatures(featureMeasurementGrid):
    featureMeasurementGrid = featureMeasurementGrid.copy()

    rankColumns = ["C1", "C2", "C3", "C4", "C5"]
    uniqueRankCountList = []
    rankRangeList = []

    for _, cardRow in featureMeasurementGrid.iterrows():
        cardRanks = sorted(cardRow[rankColumns].tolist())

        rankRange = max(cardRanks) - min(cardRanks)
        uniqueRankCount = int(len(set(cardRanks)))

        rankRangeList.append(rankRange)
        uniqueRankCountList.append(uniqueRankCount)

    featureMeasurementGrid["Unique_Rank_Count"] = uniqueRankCountList
    featureMeasurementGrid["Rank_Range"] = rankRangeList

    return featureMeasurementGrid

# Split the data into training and testing groups before any resampling.
def BuildTrainingAndTestingSets(featureMeasurementGrid, classLabelSeries):
    return train_test_split(
        featureMeasurementGrid,

```

```

classLabelSeries,
test_size=0.2,
random_state=42,
stratify=classLabelSeries
)

# Balance only the training class distribution with the same SMOTE settings.
def BalanceTrainingClassDistributionWithSmote(trainingFeatureMeasurements, trainingClassLabels):
    smoteResampler = SMOTE(k_neighbors=2, random_state=42)
    resampledTrainingFeatureMeasurements, resampledTrainingClassLabels = smoteResampler.fit_resample(
        trainingFeatureMeasurements,
        trainingClassLabels
    )

    resampledTrainingFeatureMeasurements = resampledTrainingFeatureMeasurements.astype(np.float32)
    return resampledTrainingFeatureMeasurements, resampledTrainingClassLabels

# Build class weights for imbalance handling.
def BuildClassWeights(trainingClassLabels):
    classCounts = trainingClassLabels.value_counts().to_dict()
    totalSamples = len(trainingClassLabels)
    classWeights = {cls: totalSamples / (len(classCounts) * count) for cls, count in classCounts.items()}
    return classWeights

# Scale the training data, then scale the testing data with the fitted scaler.
def ScaleTrainingAndTestingMeasurements(trainingFeatureMeasurements, testingFeatureMeasurements):
    featureScaler = StandardScaler(copy=False)

    scaledTrainingFeatureMeasurements =
featureScaler.fit_transform(trainingFeatureMeasurements.astype(np.float32))
    scaledTestingFeatureMeasurements = featureScaler.transform(testingFeatureMeasurements.astype(np.float32))

    return scaledTrainingFeatureMeasurements, scaledTestingFeatureMeasurements, featureScaler

# Build one-hot encoded versions of the training and testing data.
def BuildOneHotEncodedTrainingAndTestingMeasurements(trainingFeatureMeasurements,
testingFeatureMeasurements):
    try:
        oneHotEncoder = OneHotEncoder(handle_unknown="ignore", sparse_output=True, dtype=np.float32)
    except TypeError:
        oneHotEncoder = OneHotEncoder(handle_unknown="ignore", sparse=True, dtype=np.float32)

    oneHotTrainingFeatureMeasurements = oneHotEncoder.fit_transform(trainingFeatureMeasurements)
    oneHotTestingFeatureMeasurements = oneHotEncoder.transform(testingFeatureMeasurements)

    return oneHotTrainingFeatureMeasurements, oneHotTestingFeatureMeasurements, oneHotEncoder

# Reduce the scaled measurements with the same LDA logic.
def ProjectTrainingAndTestingMeasurementsIntoLdaSpace(
    scaledTrainingFeatureMeasurements,
    scaledTestingFeatureMeasurements,
    trainingClassLabels
):
    ldaComponentCount = min(len(set(trainingClassLabels)) - 1, scaledTrainingFeatureMeasurements.shape[1])
    ldaProjector = LDA(n_components=ldaComponentCount)

```

```

IdaTrainingFeatureMeasurements = IdaProjector.fit_transform(
    scaledTrainingFeatureMeasurements,
    trainingClassLabels
).astype(np.float32)

IdaScaler = StandardScaler()
IdaTrainingFeatureMeasurements = IdaScaler.fit_transform(IdaTrainingFeatureMeasurements)

IdaTestingFeatureMeasurements = IdaProjector.transform(scaledTestingFeatureMeasurements).astype(np.float32)
IdaTestingFeatureMeasurements = IdaScaler.transform(IdaTestingFeatureMeasurements)

return IdaTrainingFeatureMeasurements, IdaTestingFeatureMeasurements, IdaProjector

# Create a clean results folder for every saved figure.
def BuildResultsFolder():
    resultsFolder = Path("results")
    resultsFolder.mkdir(exist_ok=True)
    return resultsFolder

# Save a finished figure with a consistent figure number and caption.
def SaveFinishedFigure(figurePaper, resultsFolder, figureNumber, figureTitle, figureCaption):
    cleanedFigureCaption = " ".join(figureCaption.strip().split()).rstrip(" ")
    captionText = f"Figure {int(figureNumber)}. {cleanedFigureCaption}."
    figurePaper.subplots_adjust(bottom=0.18)
    figurePaper.text(0.5, 0.02, captionText, ha="center", va="bottom", fontsize=10)

    safeFigureStem = f"Figure_{figureNumber:02d}_{figureTitle.replace(' ', '_').replace('/', '_')}"
    figurePaper.savefig(resultsFolder / f"{safeFigureStem}.png", dpi=300, bbox_inches="tight")

# Save a finished table image with a consistent table number and caption.
def SaveFinishedTable(tablePaper, resultsFolder, tableNumber, tableTitle, tableCaption):
    cleanedTableCaption = " ".join(tableCaption.strip().split()).rstrip(" ")
    captionText = f"Table {int(tableNumber)}. {cleanedTableCaption}."
    tablePaper.subplots_adjust(bottom=0.2)
    tablePaper.text(0.5, 0.02, captionText, ha="center", va="bottom", fontsize=10)

    safeTableStem = f"Table_{tableNumber:02d}_{tableTitle.replace(' ', '_').replace('/', '_')}"
    tablePaper.savefig(resultsFolder / f"{safeTableStem}.png", dpi=300, bbox_inches="tight")

# Plot the original class distribution before SMOTE.
def PlotOriginalClassDistribution(classLabelSeries, resultsFolder, figureNumber):
    figurePaper, drawingArea = plt.subplots(figsize=(10, 5))

    classCounts = classLabelSeries.value_counts().sort_index()
    drawingArea.bar(classCounts.index.astype(str), classCounts.values)

    drawingArea.set_title("Original Poker Hand Class Distribution")
    drawingArea.set_xlabel("Class Label")
    drawingArea.set_ylabel("Count")
    drawingArea.grid(True, axis="y")

    figurePaper.tight_layout()
    SaveFinishedFigure(
        figurePaper,

```

```

    resultsFolder,
    figureNumber,
    "Original_Class_Distribution",
    "Original Poker Hand class distribution"
)
plt.show()
plt.close(figurePaper)

```

Plot the balanced training class distribution after SMOTE.

```

def PlotBalancedTrainingClassDistribution(resampledTrainingClassLabels, resultsFolder, figureNumber):
    figurePaper, drawingArea = plt.subplots(figsize=(10, 5))

```

```

    classCounts = pd.Series(resampledTrainingClassLabels).value_counts().sort_index()
    drawingArea.bar(classCounts.index.astype(str), classCounts.values)

```

```

    drawingArea.set_title("Balanced Training Class Distribution After SMOTE")
    drawingArea.set_xlabel("Class Label")
    drawingArea.set_ylabel("Count")
    drawingArea.grid(True, axis="y")

```

```

    figurePaper.tight_layout()
    SaveFinishedFigure(
        figurePaper,
        resultsFolder,
        figureNumber,
        "Balanced_Training_Class_Distribution",
        "Balanced Poker Hand training class distribution after SMOTE"
    )
    plt.show()
    plt.close(figurePaper)

```

Plot a correlation matrix for the original feature columns.

```

def PlotFeatureCorrelationMatrix(featureMeasurementGrid, resultsFolder, figureNumber):
    correlationMatrix = featureMeasurementGrid.corr()

```

```

    figurePaper, drawingArea = plt.subplots(figsize=(10, 8))
    heatMapImage = drawingArea.imshow(correlationMatrix, cmap="coolwarm", aspect="auto", vmin=-1, vmax=1)

```

```

    drawingArea.set_title("Poker Hand Feature Correlation Matrix")
    drawingArea.set_xticks(range(len(correlationMatrix.columns)))
    drawingArea.set_yticks(range(len(correlationMatrix.columns)))
    drawingArea.set_xticklabels(correlationMatrix.columns, rotation=45, ha="right")
    drawingArea.set_yticklabels(correlationMatrix.columns)

```

```

    colorBar = figurePaper.colorbar(heatMapImage, ax=drawingArea)
    colorBar.set_label("Correlation")

```

```

    figurePaper.tight_layout()
    SaveFinishedFigure(
        figurePaper,
        resultsFolder,
        figureNumber,
        "Feature_Correlation_Matrix",
        "Poker Hand feature correlation matrix"
    )
    plt.show()
    plt.close(figurePaper)

```

Plot the first two LDA directions to show class separation after reduction.

def PlotFirstTwoLdaDirections(ldaTrainingFeatureMeasurements, trainingClassLabels, resultsFolder, figureNumber):

if ldaTrainingFeatureMeasurements.shape[1] < 2:

print("LDA produced fewer than 2 components, so the first-two-directions plot was skipped.")

return

sampleSize = 5000

randomNumberGenerator = np.random.default_rng(42)

selectedSampleSize = min(sampleSize, len(ldaTrainingFeatureMeasurements))

selectedRowIndexes = randomNumberGenerator.choice(

len(ldaTrainingFeatureMeasurements),

size=selectedSampleSize,

replace=False

)

xCoordinates = ldaTrainingFeatureMeasurements[selectedRowIndexes, 0]

yCoordinates = ldaTrainingFeatureMeasurements[selectedRowIndexes, 1]

classLabels = np.array(trainingClassLabels)[selectedRowIndexes]

xCoordinates = xCoordinates + randomNumberGenerator.normal(0, 0.02, size=xCoordinates.shape)

yCoordinates = yCoordinates + randomNumberGenerator.normal(0, 0.02, size=yCoordinates.shape)

figurePaper, drawingArea = plt.subplots(figsize=(9, 6))

uniqueClassNumbers = np.unique(classLabels)

colorPicture = drawingArea.scatter(

xCoordinates,

yCoordinates,

c=classLabels,

cmap="tab10",

s=10,

alpha=0.4

)

drawingArea.set_title("Poker Hand LDA First Two Directions")

drawingArea.set_xlabel("LD1")

drawingArea.set_ylabel("LD2")

drawingArea.grid(True)

colorBar = figurePaper.colorbar(colorPicture, ax=drawingArea)

colorBar.set_label("Class Label")

colorBar.set_ticks(uniqueClassNumbers)

figurePaper.tight_layout()

SaveFinishedFigure(

figurePaper,

resultsFolder,

figureNumber,

"LDA_First_Two_Directions",

"First two LDA directions for Poker Hand training samples"

)

plt.show()

plt.close(figurePaper)

Plot the first two t-SNE directions from one-hot features for visualization only.

```

def PlotFirstTwoTsneDirectionsFromOneHot(oneHotTrainingFeatureMeasurements, trainingClassLabels,
resultsFolder, figureNumber):
    denseOneHotTrainingFeatureMeasurements = oneHotTrainingFeatureMeasurements.toarray()

    maximumVisualizationSampleCount = 6000
    if denseOneHotTrainingFeatureMeasurements.shape[0] > maximumVisualizationSampleCount:
        randomNumberGenerator = np.random.default_rng(42)
        selectedRowIndexes = randomNumberGenerator.choice(
            denseOneHotTrainingFeatureMeasurements.shape[0],
            size=maximumVisualizationSampleCount,
            replace=False
        )
        denseOneHotTrainingFeatureMeasurements =
denseOneHotTrainingFeatureMeasurements[selectedRowIndexes]
        trainingClassLabels = np.array(trainingClassLabels)[selectedRowIndexes]

    tsneProjector = TSNE(
        n_components=2,
        perplexity=30,
        learning_rate="auto",
        init="pca",
        random_state=42
    )

    tsneTrainingFeatureMeasurements =
tsneProjector.fit_transform(denseOneHotTrainingFeatureMeasurements).astype(np.float32)

    figurePaper, drawingArea = plt.subplots(figsize=(9, 6))

    uniqueClassNumbers = np.unique(trainingClassLabels)
    colorPicture = drawingArea.scatter(
        tsneTrainingFeatureMeasurements[:, 0],
        tsneTrainingFeatureMeasurements[:, 1],
        c=trainingClassLabels,
        cmap="tab10",
        s=18,
        alpha=0.75
    )

    drawingArea.set_title("Poker Hand One-Hot t-SNE First Two Directions")
    drawingArea.set_xlabel("t-SNE 1")
    drawingArea.set_ylabel("t-SNE 2")
    drawingArea.grid(True)

    colorBar = figurePaper.colorbar(colorPicture, ax=drawingArea)
    colorBar.set_label("Class Label")
    colorBar.set_ticks(uniqueClassNumbers)

    figurePaper.tight_layout()
    SaveFinishedFigure(
        figurePaper,
        resultsFolder,
        figureNumber,
        "One_Hot_TSNE_First_Two_Directions",
        "First two t-SNE directions from one-hot Poker Hand training samples"
    )
    plt.show()
    plt.close(figurePaper)

```

Train an XGBoost classifier on the given measurements and return predictions.

```
def RunXgboostClassification(trainingFeatureMeasurements, testingFeatureMeasurements, trainingClassLabels):
    classWeights = BuildClassWeights(trainingClassLabels)
    sampleWeights = np.array([classWeights[label] for label in trainingClassLabels])

    xgboostClassifier = XGBClassifier(
        objective="multi:softmax",
        num_class=len(np.unique(trainingClassLabels)),
        n_estimators=300,
        max_depth=6,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        min_child_weight=3,
        gamma=0.1,
        reg_lambda=1.0,
        reg_alpha=0.1,
        tree_method="hist",
        n_jobs=-1,
        random_state=42,
        eval_metric="mlogloss"
    )

    xgboostClassifier.fit(trainingFeatureMeasurements, trainingClassLabels, sample_weight=sampleWeights)
    predictedTestingLabels = xgboostClassifier.predict(testingFeatureMeasurements)
    return xgboostClassifier, predictedTestingLabels
```

Train a linear SVM on the given measurements and return predictions.

```
def RunLinearSvmClassification(trainingFeatureMeasurements, testingFeatureMeasurements, trainingClassLabels):
    linearSupportVectorMachine = SVC(kernel="linear", cache_size=1000)
    linearSupportVectorMachine.fit(trainingFeatureMeasurements, trainingClassLabels)
    predictedTestingLabels = linearSupportVectorMachine.predict(testingFeatureMeasurements)
    return linearSupportVectorMachine, predictedTestingLabels
```

Train an RBF SVM on the given measurements and return predictions.

```
def RunRbfSvmClassification(trainingFeatureMeasurements, testingFeatureMeasurements, trainingClassLabels):
    radialBasisSupportVectorMachine = SVC(kernel="rbf", cache_size=1000)
    radialBasisSupportVectorMachine.fit(trainingFeatureMeasurements, trainingClassLabels)
    predictedTestingLabels = radialBasisSupportVectorMachine.predict(testingFeatureMeasurements)
    return radialBasisSupportVectorMachine, predictedTestingLabels
```

Train a Random Forest classifier on the given measurements and return predictions.

```
def RunRandomForestClassification(trainingFeatureMeasurements, testingFeatureMeasurements,
trainingClassLabels):
    randomForestClassifier = RandomForestClassifier(
        n_estimators=200,
        max_depth=None,
        min_samples_split=2,
        min_samples_leaf=1,
        n_jobs=-1,
        random_state=42
    )

    randomForestClassifier.fit(trainingFeatureMeasurements, trainingClassLabels)
    predictedTestingLabels = randomForestClassifier.predict(testingFeatureMeasurements)

    return randomForestClassifier, predictedTestingLabels
```

```

# Measure plain classification accuracy for a predicted label set.
def MeasureClassificationAccuracy(testingClassLabels, predictedTestingLabels):
    return accuracy_score(testingClassLabels, predictedTestingLabels)

# Draw and save a confusion matrix heat map.
def PlotConfusionMatrixHeatMap(
    testingClassLabels,
    predictedTestingLabels,
    chartTitle,
    resultsFolder,
    figureNumber,
    figureCaption
):
    classNumbersInOrder = np.unique(np.concatenate([np.array(testingClassLabels),
np.array(predictedTestingLabels)]))
    confusionMatrixGrid = confusion_matrix(
        testingClassLabels,
        predictedTestingLabels,
        labels=classNumbersInOrder
    )

    figurePaper, drawingArea = plt.subplots(figsize=(8, 6))
    heatMapImage = drawingArea.imshow(confusionMatrixGrid, cmap="Blues", aspect="auto")

    drawingArea.set_title(chartTitle)
    drawingArea.set_xlabel("Predicted Class")
    drawingArea.set_ylabel("True Class")
    drawingArea.set_xticks(range(len(classNumbersInOrder)))
    drawingArea.set_yticks(range(len(classNumbersInOrder)))
    drawingArea.set_xticklabels(classNumbersInOrder)
    drawingArea.set_yticklabels(classNumbersInOrder)

    for rowIndex in range(confusionMatrixGrid.shape[0]):
        for columnIndex in range(confusionMatrixGrid.shape[1]):
            drawingArea.text(
                columnIndex,
                rowIndex,
                confusionMatrixGrid[rowIndex, columnIndex],
                ha="center",
                va="center",
                color="black"
            )

    colorBar = figurePaper.colorbar(heatMapImage, ax=drawingArea)
    colorBar.set_label("Count")

    figurePaper.tight_layout()
    SaveFinishedFigure(figurePaper, resultsFolder, figureNumber, chartTitle, figureCaption)
    plt.show()
    plt.close(figurePaper)

# Compare model accuracies in one clean plot.
def PlotModelAccuracyComparison(modelAccuracyBook, chartTitle, resultsFolder, figureNumber, figureFileName,
figureCaption):
    figurePaper, drawingArea = plt.subplots(figsize=(8, 5))

```

```

modelNames = list(modelAccuracyBook.keys())
modelAccuracies = list(modelAccuracyBook.values())

drawingArea.bar(modelNames, modelAccuracies)
drawingArea.set_title(chartTitle)
drawingArea.set_xlabel("Model")
drawingArea.set_ylabel("Classification Accuracy")
drawingArea.set_ylim(0, 1)
drawingArea.grid(True, axis="y")

for modelIndex, modelAccuracy in enumerate(modelAccuracies):
    drawingArea.text(
        modelIndex,
        modelAccuracy + 0.01,
        f"{modelAccuracy:.4f}",
        ha="center"
    )

figurePaper.tight_layout()
SaveFinishedFigure(figurePaper, resultsFolder, figureNumber, figureFileName, figureCaption)
plt.show()
plt.close(figurePaper)

# Save a compact results table.
def SaveModelAccuracyTable(modelAccuracyBook, resultsFolder):
    accuracySummaryTable = pd.DataFrame(
        {
            "Model": list(modelAccuracyBook.keys()),
            "Classification Accuracy": list(modelAccuracyBook.values())
        }
    )

    accuracySummaryTable.to_csv(resultsFolder /
    "Table_1_Model_Accuracy_Summary_Raw_LDA_And_One_Hot.csv", index=False)
    return accuracySummaryTable

# Plot and save the model accuracy summary as a captioned table image.
def PlotModelAccuracySummaryTable(accuracySummaryTable, resultsFolder, tableNumber, tableTitle, tableCaption):
    tablePaper, drawingArea = plt.subplots(figsize=(10, 5))
    drawingArea.axis("off")

    displayTable = accuracySummaryTable.copy()
    displayTable["Classification Accuracy"] = displayTable["Classification Accuracy"].map(lambda value: f"{value:.4f}")

    renderedTable = drawingArea.table(
        cellText=displayTable.values,
        colLabels=displayTable.columns,
        cellLoc="center",
        loc="center"
    )
    renderedTable.auto_set_font_size(False)
    renderedTable.set_fontsize(10)
    renderedTable.scale(1, 1.25)

    tablePaper.tight_layout()
    SaveFinishedTable(tablePaper, resultsFolder, tableNumber, tableTitle, tableCaption)
    plt.show()

```

```

plt.close(tablePaper)

# Pick the best result for a single model across raw, LDA, and one-hot inputs.
def ChooseBestModelVersion(
    rawAccuracy,
    rawPredictedTestingLabels,
    ldaAccuracy,
    ldaPredictedTestingLabels,
    oneHotAccuracy,
    oneHotPredictedTestingLabels
):
    if oneHotAccuracy >= ldaAccuracy and oneHotAccuracy >= rawAccuracy:
        return oneHotAccuracy, oneHotPredictedTestingLabels, "With One-Hot"
    if ldaAccuracy >= rawAccuracy:
        return ldaAccuracy, ldaPredictedTestingLabels, "With LDA"
    return rawAccuracy, rawPredictedTestingLabels, "Baseline"

# Run the full Poker Hand study from start to finish.
def RunPokerHandClassificationStudy():
    resultsFolder = BuildResultsFolder()

    pokerHandDataFrame = LoadPokerHandDatasetFromCsvFile("poker-hand-training-true.data")
    originalFeatureMeasurementGrid, originalClassLabelSeries =
SplitFeatureColumnsAndClassLabels(pokerHandDataFrame)
    originalFeatureMeasurementGrid = BuildDomainKnowledgeFeatures(originalFeatureMeasurementGrid)

    PlotOriginalClassDistribution(originalClassLabelSeries, resultsFolder, 1)
    PlotFeatureCorrelationMatrix(originalFeatureMeasurementGrid, resultsFolder, 2)

    (
        trainingFeatureMeasurements,
        testingFeatureMeasurements,
        trainingClassLabels,
        testingClassLabels
    ) = BuildTrainingAndTestingSets(
        originalFeatureMeasurementGrid,
        originalClassLabelSeries
    )

    (
        resampledTrainingFeatureMeasurements,
        resampledTrainingClassLabels
    ) = BalanceTrainingClassDistributionWithSmote(
        trainingFeatureMeasurements,
        trainingClassLabels
    )

    PlotBalancedTrainingClassDistribution(resampledTrainingClassLabels, resultsFolder, 3)

    print("Training set:", resampledTrainingFeatureMeasurements.shape)
    print("Testing set:", testingFeatureMeasurements.shape)

    (
        scaledTrainingFeatureMeasurements,
        scaledTestingFeatureMeasurements,
        featureScaler
    ) = ScaleTrainingAndTestingMeasurements(

```

```

    resampledTrainingFeatureMeasurements,
    testingFeatureMeasurements
)

(
    oneHotTrainingFeatureMeasurements,
    oneHotTestingFeatureMeasurements,
    oneHotEncoder
) = BuildOneHotEncodedTrainingAndTestingMeasurements(
    resampledTrainingFeatureMeasurements,
    testingFeatureMeasurements
)

(
    ldaTrainingFeatureMeasurements,
    ldaTestingFeatureMeasurements,
    ldaProjector
) = ProjectTrainingAndTestingMeasurementsIntoLdaSpace(
    scaledTrainingFeatureMeasurements,
    scaledTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

print("Resampled training:", resampledTrainingFeatureMeasurements.shape)
print("LDA training:", ldaTrainingFeatureMeasurements.shape)
print("One-Hot training:", oneHotTrainingFeatureMeasurements.shape)
print("Testing set:", testingFeatureMeasurements.shape)
print("LDA testing:", ldaTestingFeatureMeasurements.shape)
print("One-Hot testing:", oneHotTestingFeatureMeasurements.shape)

PlotFirstTwoLdaDirections(ldaTrainingFeatureMeasurements, resampledTrainingClassLabels, resultsFolder, 4)
PlotFirstTwoTsneDirectionsFromOneHot(oneHotTrainingFeatureMeasurements, resampledTrainingClassLabels,
resultsFolder, 5)

(
    rawXgboostClassifier,
    rawXgboostPredictedTestingLabels
) = RunXgboostClassification(
    scaledTrainingFeatureMeasurements,
    scaledTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

(
    rawLinearSupportVectorMachine,
    rawLinearPredictedTestingLabels
) = RunLinearSvmClassification(
    scaledTrainingFeatureMeasurements,
    scaledTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

(
    rawRadialBasisSupportVectorMachine,
    rawRadialBasisPredictedTestingLabels
) = RunRbfSvmClassification(
    scaledTrainingFeatureMeasurements,
    scaledTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

```

```

(
    rawRandomForestClassifier,
    rawRandomForestPredictedTestingLabels
) = RunRandomForestClassification(
    scaledTrainingFeatureMeasurements,
    scaledTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

rawXgboostAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    rawXgboostPredictedTestingLabels
)

rawRadialBasisSvmAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    rawRadialBasisPredictedTestingLabels
)

rawLinearSvmAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    rawLinearPredictedTestingLabels
)

rawRandomForestAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    rawRandomForestPredictedTestingLabels
)

rawModelAccuracyBook = {
    "Linear SVM": rawLinearSvmAccuracy,
    "RBF SVM": rawRadialBasisSvmAccuracy,
    "XGBoost": rawXgboostAccuracy,
    "Random Forest": rawRandomForestAccuracy
}

PlotModelAccuracyComparison(
    rawModelAccuracyBook,
    "Model Accuracy Comparison",
    resultsFolder,
    6,
    "Model_Accuracy_Comparison",
    "Model accuracy comparison on baseline scaled features"
)

(
    IdaXgboostClassifier,
    IdaXgboostPredictedTestingLabels
) = RunXgboostClassification(
    IdaTrainingFeatureMeasurements,
    IdaTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

(
    IdaRandomForestClassifier,
    IdaRandomForestPredictedTestingLabels
) = RunRandomForestClassification(
    IdaTrainingFeatureMeasurements,
    IdaTestingFeatureMeasurements,

```

```

    resampledTrainingClassLabels
)

(
    IdaLinearSupportVectorMachine,
    IdaLinearPredictedTestingLabels
) = RunLinearSvmClassification(
    IdaTrainingFeatureMeasurements,
    IdaTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

(
    IdaRadialBasisSupportVectorMachine,
    IdaRadialBasisPredictedTestingLabels
) = RunRbfSvmClassification(
    IdaTrainingFeatureMeasurements,
    IdaTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

IdaXgboostAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    IdaXgboostPredictedTestingLabels
)

IdaRadialBasisSvmAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    IdaRadialBasisPredictedTestingLabels
)

IdaLinearSvmAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    IdaLinearPredictedTestingLabels
)

IdaRandomForestAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    IdaRandomForestPredictedTestingLabels
)

IdaModelAccuracyBook = {
    "Linear SVM": IdaLinearSvmAccuracy,
    "RBF SVM": IdaRadialBasisSvmAccuracy,
    "XGBoost": IdaXgboostAccuracy,
    "Random Forest": IdaRandomForestAccuracy
}

PlotModelAccuracyComparison(
    IdaModelAccuracyBook,
    "Model Accuracy Comparison With LDA",
    resultsFolder,
    7,
    "Model_Accuracy_Comparison_With_LDA",
    "Model accuracy comparison with LDA features"
)

(
    oneHotXgboostClassifier,
    oneHotXgboostPredictedTestingLabels

```

```

) = RunXgboostClassification(
    oneHotTrainingFeatureMeasurements,
    oneHotTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

(
    oneHotLinearSupportVectorMachine,
    oneHotLinearPredictedTestingLabels
) = RunLinearSvmClassification(
    oneHotTrainingFeatureMeasurements,
    oneHotTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

(
    oneHotRadialBasisSupportVectorMachine,
    oneHotRadialBasisPredictedTestingLabels
) = RunRbfSvmClassification(
    oneHotTrainingFeatureMeasurements,
    oneHotTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

(
    oneHotRandomForestClassifier,
    oneHotRandomForestPredictedTestingLabels
) = RunRandomForestClassification(
    oneHotTrainingFeatureMeasurements,
    oneHotTestingFeatureMeasurements,
    resampledTrainingClassLabels
)

oneHotXgboostAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    oneHotXgboostPredictedTestingLabels
)

oneHotRadialBasisSvmAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    oneHotRadialBasisPredictedTestingLabels
)

oneHotLinearSvmAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    oneHotLinearPredictedTestingLabels
)

oneHotRandomForestAccuracy = MeasureClassificationAccuracy(
    testingClassLabels,
    oneHotRandomForestPredictedTestingLabels
)

oneHotModelAccuracyBook = {
    "Linear SVM": oneHotLinearSvmAccuracy,
    "RBF SVM": oneHotRadialBasisSvmAccuracy,
    "XGBoost": oneHotXgboostAccuracy,
    "Random Forest": oneHotRandomForestAccuracy
}

```

```

PlotModelAccuracyComparison(
    oneHotModelAccuracyBook,
    "Model Accuracy Comparison With One-Hot",
    resultsFolder,
    8,
    "Model_Accuracy_Comparison_With_One_Hot",
    "Model accuracy comparison with one-hot encoded features"
)

bestXgboostAccuracy, bestXgboostPredictedTestingLabels, bestXgboostVersion = ChooseBestModelVersion(
    rawXgboostAccuracy,
    rawXgboostPredictedTestingLabels,
    ldaXgboostAccuracy,
    ldaXgboostPredictedTestingLabels,
    oneHotXgboostAccuracy,
    oneHotXgboostPredictedTestingLabels
)

bestRadialBasisSvmAccuracy, bestRadialBasisPredictedTestingLabels, bestRadialBasisSvmVersion =
ChooseBestModelVersion(
    rawRadialBasisSvmAccuracy,
    rawRadialBasisPredictedTestingLabels,
    ldaRadialBasisSvmAccuracy,
    ldaRadialBasisPredictedTestingLabels,
    oneHotRadialBasisSvmAccuracy,
    oneHotRadialBasisPredictedTestingLabels
)

bestLinearSvmAccuracy, bestLinearPredictedTestingLabels, bestLinearSvmVersion = ChooseBestModelVersion(
    rawLinearSvmAccuracy,
    rawLinearPredictedTestingLabels,
    ldaLinearSvmAccuracy,
    ldaLinearPredictedTestingLabels,
    oneHotLinearSvmAccuracy,
    oneHotLinearPredictedTestingLabels
)

bestRandomForestAccuracy, bestRandomForestPredictedTestingLabels, bestRandomForestVersion =
ChooseBestModelVersion(
    rawRandomForestAccuracy,
    rawRandomForestPredictedTestingLabels,
    ldaRandomForestAccuracy,
    ldaRandomForestPredictedTestingLabels,
    oneHotRandomForestAccuracy,
    oneHotRandomForestPredictedTestingLabels
)

print(f"Linear SVM Accuracy: {rawLinearSvmAccuracy:.4f}")
print(f"RBF SVM Accuracy: {rawRadialBasisSvmAccuracy:.4f}")
print(f"XGBoost Accuracy: {rawXgboostAccuracy:.4f}")
print(f"Random Forest Accuracy: {rawRandomForestAccuracy:.4f}")

print(f"Linear SVM Accuracy With LDA: {ldaLinearSvmAccuracy:.4f}")
print(f"RBF SVM Accuracy With LDA: {ldaRadialBasisSvmAccuracy:.4f}")
print(f"XGBoost Accuracy With LDA: {ldaXgboostAccuracy:.4f}")
print(f"Random Forest Accuracy With LDA: {ldaRandomForestAccuracy:.4f}")

print(f"Linear SVM Accuracy With One-Hot: {oneHotLinearSvmAccuracy:.4f}")
print(f"RBF SVM Accuracy With One-Hot: {oneHotRadialBasisSvmAccuracy:.4f}")
print(f"XGBoost Accuracy With One-Hot: {oneHotXgboostAccuracy:.4f}")

```

```

print(f"Random Forest Accuracy With One-Hot: {oneHotRandomForestAccuracy:.4f}")

print(f"Best XGBoost Version: {bestXgboostVersion}")
print(f"Best RBF SVM Version: {bestRadialBasisSvmVersion}")
print(f"Best Linear SVM Version: {bestLinearSvmVersion}")
print(f"Best Random Forest Version: {bestRandomForestVersion}")

PlotConfusionMatrixHeatMap(
    testingClassLabels,
    bestXgboostPredictedTestingLabels,
    f"XGBoost Confusion Matrix ({bestXgboostVersion})",
    resultsFolder,
    9,
    f"XGBoost confusion matrix using the best version ({bestXgboostVersion})"
)

PlotConfusionMatrixHeatMap(
    testingClassLabels,
    bestRadialBasisPredictedTestingLabels,
    f"RBF SVM Confusion Matrix ({bestRadialBasisSvmVersion})",
    resultsFolder,
    10,
    f"RBF SVM confusion matrix using the best version ({bestRadialBasisSvmVersion})"
)

PlotConfusionMatrixHeatMap(
    testingClassLabels,
    bestLinearPredictedTestingLabels,
    f"Linear SVM Confusion Matrix ({bestLinearSvmVersion})",
    resultsFolder,
    11,
    f"Linear SVM confusion matrix using the best version ({bestLinearSvmVersion})"
)

PlotConfusionMatrixHeatMap(
    testingClassLabels,
    bestRandomForestPredictedTestingLabels,
    f"Random Forest Confusion Matrix ({bestRandomForestVersion})",
    resultsFolder,
    12,
    f"Random Forest confusion matrix using the best version ({bestRandomForestVersion})"
)

modelAccuracyBook = {
    "Linear SVM": rawLinearSvmAccuracy,
    "RBF SVM": rawRadialBasisSvmAccuracy,
    "XGBoost": rawXgboostAccuracy,
    "Random Forest": rawRandomForestAccuracy,
    "Linear SVM With LDA": IdaLinearSvmAccuracy,
    "RBF SVM With LDA": IdaRadialBasisSvmAccuracy,
    "XGBoost With LDA": IdaXgboostAccuracy,
    "Random Forest With LDA": IdaRandomForestAccuracy,
    "Linear SVM With One-Hot": oneHotLinearSvmAccuracy,
    "RBF SVM With One-Hot": oneHotRadialBasisSvmAccuracy,
    "XGBoost With One-Hot": oneHotXgboostAccuracy,
    "Random Forest With One-Hot": oneHotRandomForestAccuracy
}

```

```

accuracySummaryTable = SaveModelAccuracyTable(modelAccuracyBook, resultsFolder)
PlotModelAccuracySummaryTable(
    accuracySummaryTable,
    resultsFolder,
    1,
    "Model_Accuracy_Summary_Raw_LDA_And_One_Hot",
    "Model accuracy summary across baseline, LDA, and one-hot feature settings"
)

print("\nAccuracy Summary")
print(accuracySummaryTable.to_string(index=False))
print(f"\nSaved outputs to: {resultsFolder.resolve()}")

if __name__ == "__main__":
    RunPokerHandClassificationStudy()

```